BCSE202L- Data Structures and Algorithm
Module 1

Dr. G. Praveen Kumar
Assistant Professor, Gr2
SMEC, SCOPE

Module:1	Algorithm Analysis	8 hours
Importance of algorithms and data structures - Fundamentals of algorithm analysis: Space and time complexity of an algorithm, Types of asymptotic notations and orders of growth - Algorithm efficiency – best case, worst case, average case - Analysis of non-recursive and recursive algorithms - Asymptotic analysis for recurrence relation: Iteration Method, Substitution Method, Master Method and Recursive Tree Method.		
Module:2	**Linear Data Structures**	**7 hours**
Arrays: 1D and 2D array- Stack - Applications of stack: Expression Evaluation, Conversion of Infix to postfix and prefix expression, Tower of Hanoi – Queue - Types of Queue: Circular Queue, Double Ended Queue (deQueue) - Applications – List: Singly linked lists, Doubly linked lists, Circular linked lists- Applications: Polynomial Manipulation.		
Module:3	**Searching and Sorting**	**7 hours**
Searching: Linear Search and binary search – Applications. Sorting: Insertion sort, Selection sort, Bubble sort, Counting sort, Quick sort, Merge sort - Analysis of sorting algorithms.		
Module:4	**Trees**	**6 hours**
Introduction - Binary Tree: Definition and Properties - Tree Traversals- Expression Trees:- Binary Search Trees - Operations in BST: insertion, deletion, finding min and max, finding the k^{th} minimum element.		

Module:5	Graphs	6 hours
Terminology – Representation of Graph – Graph Traversal: Breadth First Search (BFS), Depth First Search (DFS) - Minimum Spanning Tree: Prim's, Kruskal's - Single Source Shortest Path: Dijkstra's Algorithm.		
Module:6	**Hashing**	**4 hours**
Hash functions - Separate chaining - Open hashing: Linear probing, Quadratic probing, Double hashing - Closed hashing - Random probing – Rehashing - Extendible hashing.		
Module:7	**Heaps and AVL Trees**	**5 hours**
Heaps - Heap sort- Applications -Priority Queue using Heaps. AVL trees: Terminology, basic operations (rotation, insertion and deletion).		
Module:8	**Contemporary Issues**	**2 hours**

Text Book	
1.	Mark A. Weiss, Data Structures & Algorithm Analysis in C++, 4th Edition, 2013, Pearson Education.
Reference Books	
1.	Alfred V. Aho, Jeffrey D. Ullman and John E. Hopcroft, Data Structures and Algorithms, 1983, Pearson Education.
2.	Horowitz, Sahni and S. Anderson-Freed, Fundamentals of Data Structures in C, 2008, 2nd Edition, Universities Press.
3.	Thomas H. Cormen, C.E. Leiserson, R L. Rivest and C. Stein, Introduction to Algorithms, 2009, 3rd Edition, MIT Press.

Dr Praveen Kumar, Research Paper Publications

	2020	2022	2023	2024	2025
Published	2	1	6	10	4
Category	Q1 :2	SCI	Q1 :2, Q4 :1 SCI :1	Q1 : 9 SCI :1	Q1 :4
Publisher	2 Elsevier	1 Springer Nature	4-Elsevier, 1-Springer Nature 1-Taylor and Francis	9 Elsevier, 1 Springer Nature	4 Elsevier
Impact factor	Q1:5.1 to 8.7	-	Q1 & Q4 : 9 to 16.3	Q1: 6 to 9.9	Q1 :10.1 to 16.3
Corresponding author	1	-	5	6	3
B.Tech Students	-	-	1	2	3
PhD Scholars	-	-	5	9	1
Foreign collaborations	1	1	3	7	2
Open access	No	No	1 (VIT), without any collaboration	2 (URV, Spain, Arizona State university, USA)	Australia, S. Korea

Achievements :

- **Marie Curies Fellowship 2014, Universitat Rovira I Virgili, Spain**
- **BHAVAN Fellowship 2018, Arizona State University, USA**

Completed Projects

- **DST : Rs 1.6 Crores**
- **Bharat Benz: Rs. 10 Lacs**
- **VIT Bhopal**

Research Area (Both Simulation and Experiment)

- Digital Twin
- Hydrogen Production, Storage and Transportation
- HVAC & R
- Power cycles, Desalination
- Hyperloop (Recent work)

- Published research papers : https://scholar.google.co.in/citations?user=lnPWFYIAAAAJ&hl=en
- https://orcid.org/my-orcid?orcid=0000-0002-5608-2389

Mentors for students and their laurel achievements to VIT

1. Second runner-up in the 2023 aQUEST National Competition (Rs. 25,000 prize).
2. Winner in the 2024 Trane Technologies IANC Hackathon (Rs. 50,000 prize).
3. Second runner-up in the 2024 ISHRAE National Hackathon (Rs. 50,000 prize).
4. Winner in the 2024 Daikin Global Student Poster Competition (Rs. 25,000 prize).
5. India level Student research project grant (SRPG 2024-25) for Rs. 50,000, Jan to May 2025.

I have mentored students in over 10 events, resulting in over Rs. 3,50,000 in prize money for VIT students in various national and international competitions.

LOR Submitted to students Mar 2025

Graduate Admissions	Inbox	Vigneshkumar Venugopal Requests Letter of Recommendation for UC Santa Barbara - . Your letter of recommendation will be included in this applicant's file as one of the items evaluated to d...	Dec 11
me, Dean 3	Inbox	PhD Student Placement: Karthikeyan B - Carrier Corporation _Reg - the offer letter will be given in person upon joining on January 6, 2025. I will share it with you as soon as I receive ...	
support	Inbox	Your Letter of Recommendation is requested - as a reference. An online recommendation form is waiting for you at ASU's Letter of Recommendation Interface. When	Dec 9
Graduate College Ad.	Inbox	UA Graduate Admissions Recommendation For Sai Rishin Bussa - and has requested that you provide a Recommendation on their behalf. Sai Rishin Bussa has waived the right to view this reco...	Dec 9
Res, me 2	Inbox	Thesis Evaluation reports and Office order of Mr. ARAVINDAN M (Reg. No. 20PHD0712) - Reg - Guide is requested to raise the Oral Viva Voce meeting request in VTOP. In case of any request fr... Aravindan M_Of... Aravindan M_Fo... Aravindan M_In... +2	Dec 7
Cornell University .	Inbox	**Online recommendation request for Someshvar Nirmalbabu** - and has requested that you submit a recommendation on their behalf. Please confer with the applicant or visit our Fields	**Dec 5**
CU Denver Office of.	Inbox	Recommendation Request - CU Denver - provide a recommendation letter. If you are willing to write a recommendation, click this link to access a secure site to	Dec 5
MIT Graduate Admiss.	Inbox	Submit your MIT Graduate letter of recommendation for Someshvar Nirmalbabu - Nirmalbabu has requested a letter of recommendation to support their application for admission to the Mech...	Dec 4
The University of T.	Inbox	The University of Texas at Austin Graduate School - Recommendation Request -) has requested that you write a letter of recommendation to the Graduate School at The University of Texas at ...	Dec 4
UCI Graduate Admiss.	Inbox	Recommendation Request from Devesh Donthi Rajesh Kumar for University of California, Irvine Graduate Division - Kumar has requested that you write a letter of recommendation to University...	Dec 4
Stony Brook Univers.	Inbox	Stony Brook University Application Recommendation Request: Devesh Donthi Rajesh Kumar - Kumar has requested that you write a letter of recommendation to Stony Brook University's Comp...	Dec 4
support	Inbox	You have received a recommendation request from an applicant via EngineeringCAS - and is requesting an online recommendation from you. Devesh Donthi Rajesh Kumar provided the followi...	Dec 4
Illinois Graduate A. 2	Inbox	Recommendation Request from Devesh Donthi Rajesh Kumar - Kumar has requested that you write a letter of recommendation for their Illinois graduate application to the Information	Dec 4
Purdue University G.	Inbox	Recommendation Request from Devesh Donthi Rajesh Kumar for Purdue University Graduate Admissions - Kumar has requested that you write a letter of recommendation to Purdue University ...	Dec 4
UCLA Graduate Admis.	Inbox	Recommendation Request from Devesh Donthi Rajesh Kumar for UCLA Graduate Admissions - Kumar has requested that you write a letter of recommendation on their behalf for the Computer...	Dec 4
University of Michi.	Inbox	Request from Devesh Donthi Rajesh Kumar for a Letter of Recommendation - Kumar has requested that you submit a letter of recommendation. This applicant has waived the right to view you...	Dec 4
Berkeley Graduate A.	Inbox	Recommendation request from Devesh Donthi Rajesh Kumar for the University of California, Berkeley -) has requested a letter of recommendation from you in support of an application for ad...	Dec 4
Sankaran M	Inbox	Paper Publication Details for Research Award January 2023 to December 2023. - your kind reference. These details were obtained from the monthly-wise paper publication data circulated by ...	Dec 3

LOR Submitted to students Nov 2024

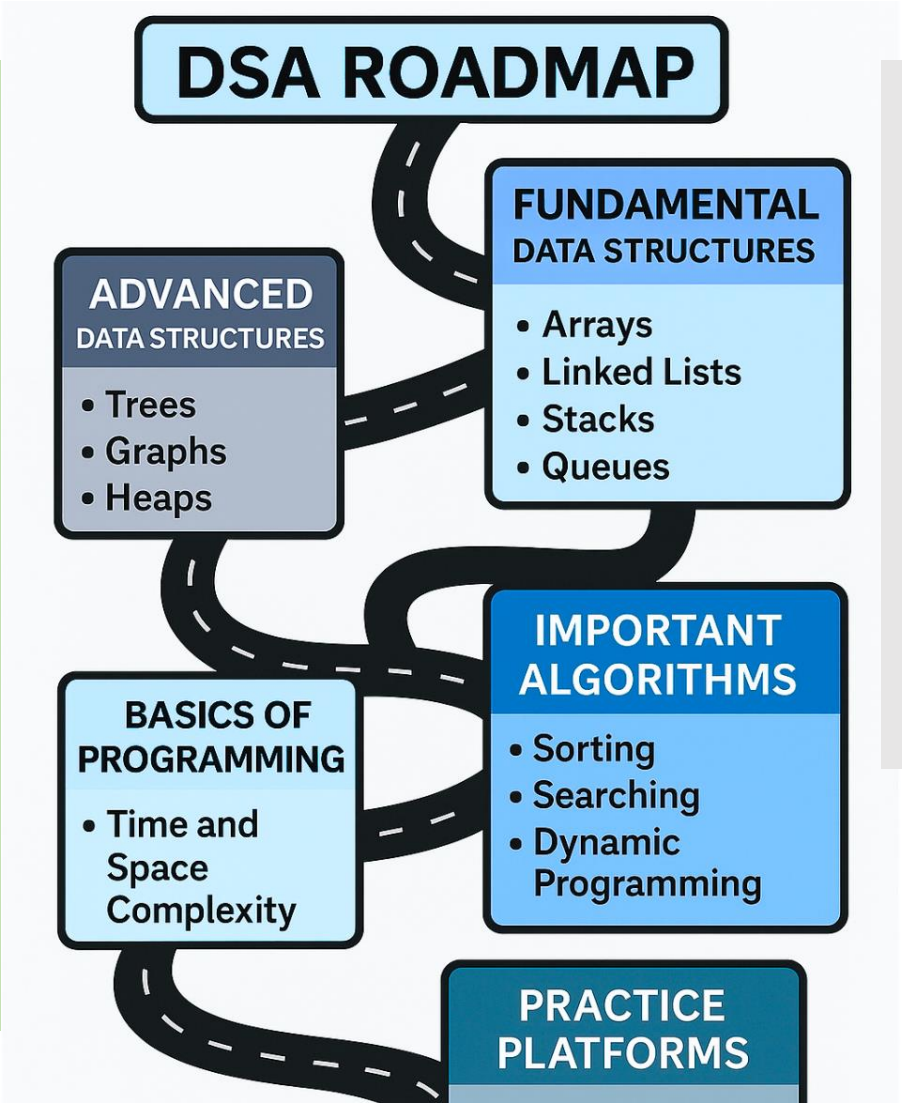
University at Buffa.	Inbox	Recommendation Request from Sai Rishin Bussa for the University at Buffalo - Bussa has requested that you write a letter of recommendation for their application to the Computer Science an...	Dec 2
me		Print out - *Regards,* *Dr. G. Praveen Kumar* Assistant Professor (sr.) | School of Mechanical Engineering ISHRAE Letter to Revok... Response to th... Annexure1.pdf	Dec 2
University of Cinci.	Inbox	Sai Rishin Bussa Requesting a Recommendation for University of Cincinnati Graduate Application - as a reference for their graduate program application to University of Cincinnati. We would g...	Dec 2
University of Flori.	Inbox	Recommendation Request - Bussa has requested that you submit a recommendation form in support of an application to University of Florida. This applicant	Dec 2
UT Dallas Admission.	Inbox	The University of Texas at Dallas Recommendation Request - and is requesting you to complete a letter of recommendation on their behalf. Please complete the following form to be included	Dec 2
Northeastern Univer.	Inbox	Recommendation Request from Sai Rishin Bussa for Northeastern University - Bussa has requested that you write a letter of recommendation to Northeastern University on their behalf. In an e...	Dec 2
University of Bridg.	Inbox	Recommendation Request for Someshvar N Nirmalbabu - Nirmalbabu has requested that you write a letter of recommendation to University of Bridgeport on their behalf. In an effort	Dec 1
Jianzhong Wu	Inbox	Reviewer Notification of Editor Decision - author decision letter and reviewer reports can be found below. We appreciate your time and effort in reviewing this paper	Nov 30
Praveen, me 2	Inbox	Fwd: Johns Hopkins- USA-Whiting School of Engineering (WSE)-WSE Dean's Master's Fellowship for up to 50 admitted students - to three letters of recommendation, a statement of purpose, ...	Nov 30
The University of T.	Inbox	The University of Texas at Austin Graduate School - Recommendation Request -) has requested that you write a letter of recommendation to the Graduate School at The University of Texas at ...	Nov 30
NYU Tandon Graduate	Inbox	NYU Tandon School of Engineering Recommendation Request for Someshvar Nirmalbabu - provide a recommendation. If you are uploading a letter of recommendation, NYU Tandon's Office of...	Nov 30
The University of T.	Inbox	The University of Texas at Austin Graduate School - Recommendation Request -) has requested that you write a letter of recommendation to the Graduate School at The University of Texas at ...	Nov 29
support	Inbox	You have an Indiana University Graduate CAS recommendation request - and is requesting an online recommendation from you. Sai Rishin Bussa provided the following comments with this req...	Nov 29
Northeastern Univer.	Inbox	Recommendation Request from Devesh Donthi Rajesh Kumar for Northeastern University - Kumar has requested that you write a letter of recommendation to Northeastern University on their ...	Nov 29
NYU Tandon Graduate	Inbox	NYU Tandon School of Engineering Recommendation Request for Devesh Donthi Rajesh Kumar - provide a recommendation. If you are uploading a letter of recommendation, NYU Tandon's Off...	Nov 29
NYU Tandon Graduate 2	Inbox	NYU Tandon School of Engineering Recommendation Request for Someshvar Nirmalbabu - provide a recommendation. If you are uploading a letter of recommendation, NYU Tandon's Office of...	Nov 28
Stony Brook Univers.	Inbox	Stony Brook University Application Recommendation Request: Sai Rishin Bussa - Bussa has requested that you write a letter of recommendation to Stony Brook University's Computer Science ...	Nov 28
Ramadas, me 20	Inbox	Maitri Fellow - for your reference. Since this process is new for them, I have visited the HR office multiple times to follow up. *Basic salary	Nov 28

Context :

Importance of algorithms and data structures - Fundamentals of algorithm analysis: Space and time complexity of an algorithm, Types of asymptotic notations and orders of growth

Data Structures

- These are specialized formats for organizing, processing, retrieving, and storing data.
- Common examples include arrays, linked lists, stacks, queues, trees, and graphs.
- Each structure is designed for specific data types and operations, offering various trade-offs in terms of performance and memory usage.

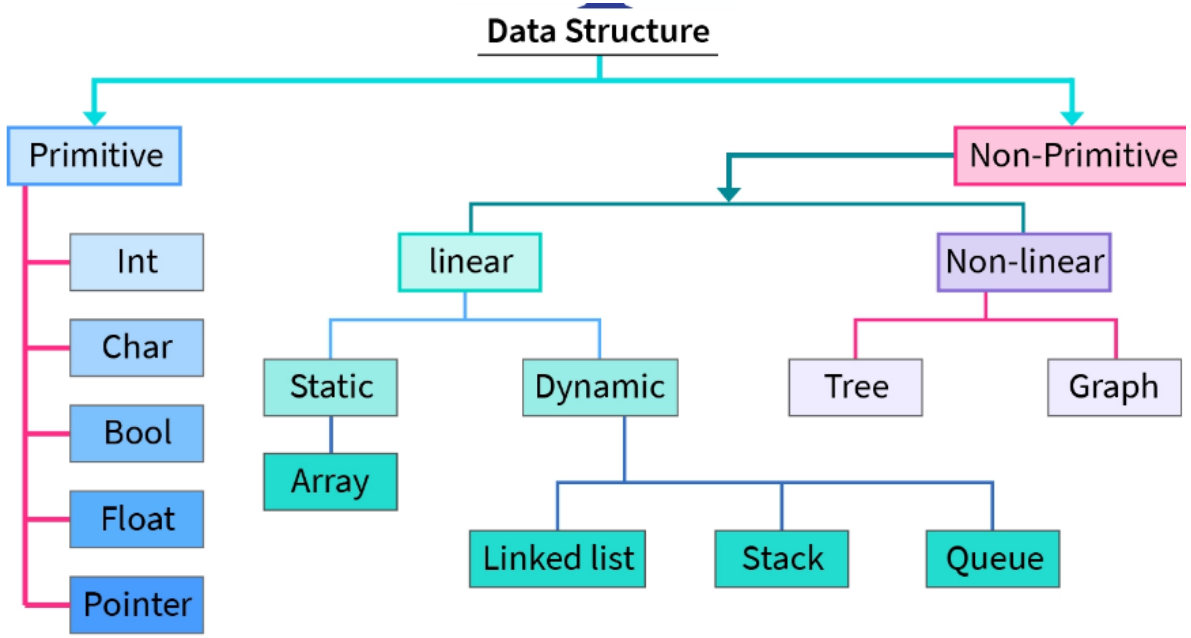


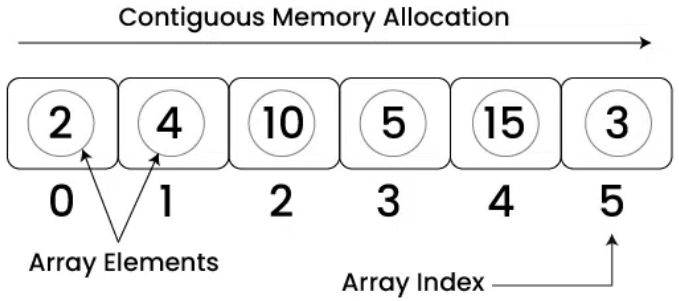
Algorithms

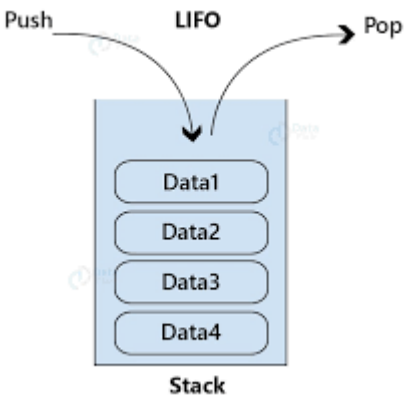
- These are a set of well-defined, step-by-step instructions designed to perform a specific task or solve a particular problem.
- Algorithms operate on data, and the choice of a data structure can significantly impact the efficiency of an algorithm.

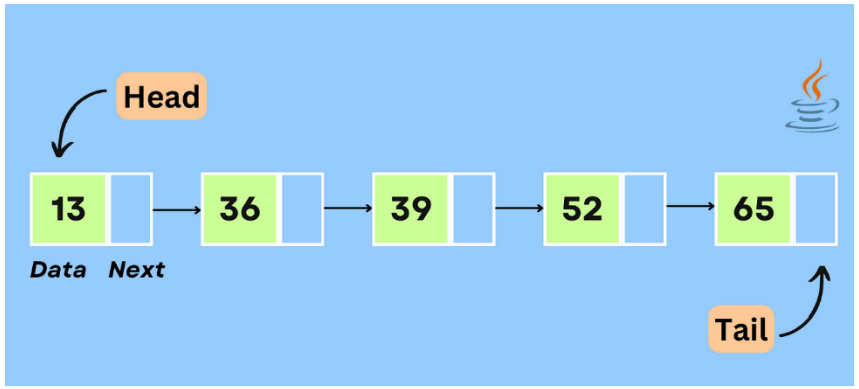
Data Structures

- A **Data Structure** is a specific way of organizing, storing, and managing data in a computer so that it can be accessed and modified efficiently. Different data structures are suited for different kinds of applications.









1. Primitive Data Structures

Primitive data structures are the most basic types of data structures that are directly supported by most programming languages. They operate closely with the machine-level instructions.

Data Type	Description	Example (in C/Java/Python)
int	Stores integer values	int age = 25;
char	Stores a single character	char grade = 'A';
float	Stores decimal numbers	float pi = 3.14;
bool	Stores boolean values (T/F)	bool isOpen = true;
string	Stores a sequence of characters	string name = “Data";

```
a = 10          # int
b = 'Z'         # char
c = 3.14        # float
d = True        # bool
e = "Hello"     # string
```

2. Non-Primitive Data Structures

Non-primitive data structures are more advanced structures that are derived from primitive types. They allow the storage and organization of multiple values in logical and efficient ways.

A. Linear Data Structures:

Array: Collection of elements of same type stored in contiguous memory.

Linked List: Sequence of nodes connected by pointers.

Stack: LIFO (Last-In-First-Out).

Queue: FIFO (First-In-First-Out).

B. Non-linear Data Structures:

Tree: Hierarchical structure (e.g., binary trees).

Graph: Network structure (nodes connected with edges).

```python
# Array (List in Python)
marks = [85, 90, 78]

# Stack using list
stack = []
stack.append(10)
stack.append(20)
stack.pop()   # Removes 20

# Queue using deque
from collections import deque
queue = deque([1, 2, 3])
queue.popleft()  # Removes 1
```

Importance of Algorithms and Data Structures

- Efficient problem-solving using optimized logic and storage.
- Reduces time and space usage in programs.
- Enhances scalability, speed, and performance.
- Core foundation for real-world applications (e.g., Google Search, Banking Systems).

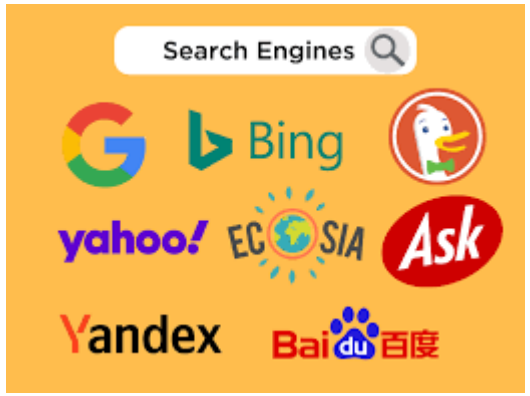
Fundamentals of Algorithm Analysis

•Measures algorithm efficiency in terms of **time and space**.

•Independent of hardware and programming languages.

•Crucial for comparing different algorithms solving the same problem.

DSA provides the knowledge to write code but also efficient in terms of time and memory.



Use Searching and sorting methods

Use graph algorithms like Dijkstra's or A* to find the shortest path

Utilize tree structures like B-Trees for efficient data retrieval.

Time and Space Complexity

Time and Space Complexity

- To evaluate the efficiency of an algorithm. Helps in selecting the most optimal solution for a problem.

Algorithm Analysis

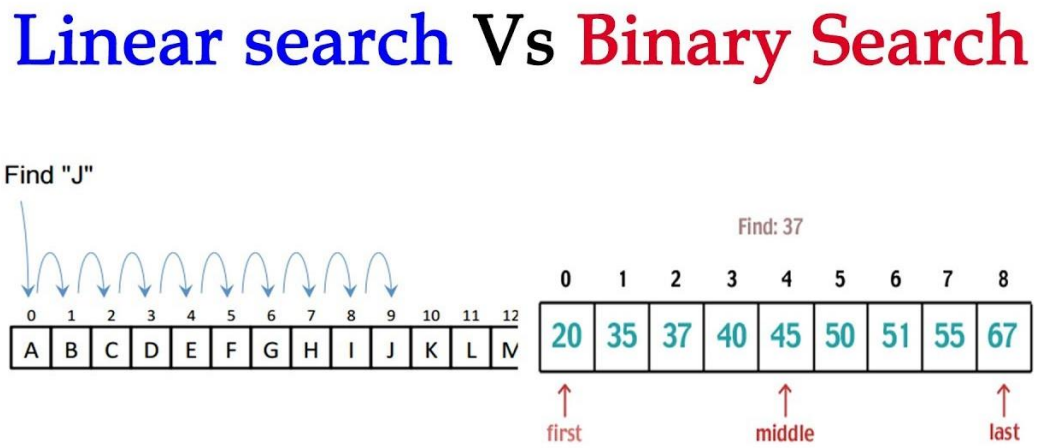
- Understanding how the algorithm behaves as the input size (n) grows.

Time Complexity

- Measures the number of operations performed as a function of input size.
- Example notations: O(1) – Constant, O(n) – Linear, $O(n^2)$ – Quadratic

Space Complexity

- Measures the amount of memory required relative to input size.
- Includes: Input storage, Auxiliary data structures, Recursive stack memory



Trade-off Between Time and Space

- Some algorithms use more memory to run faster. Others save space but take longer to execute.
- **Time Complexity:** Number of operations relative to input size (*n*).

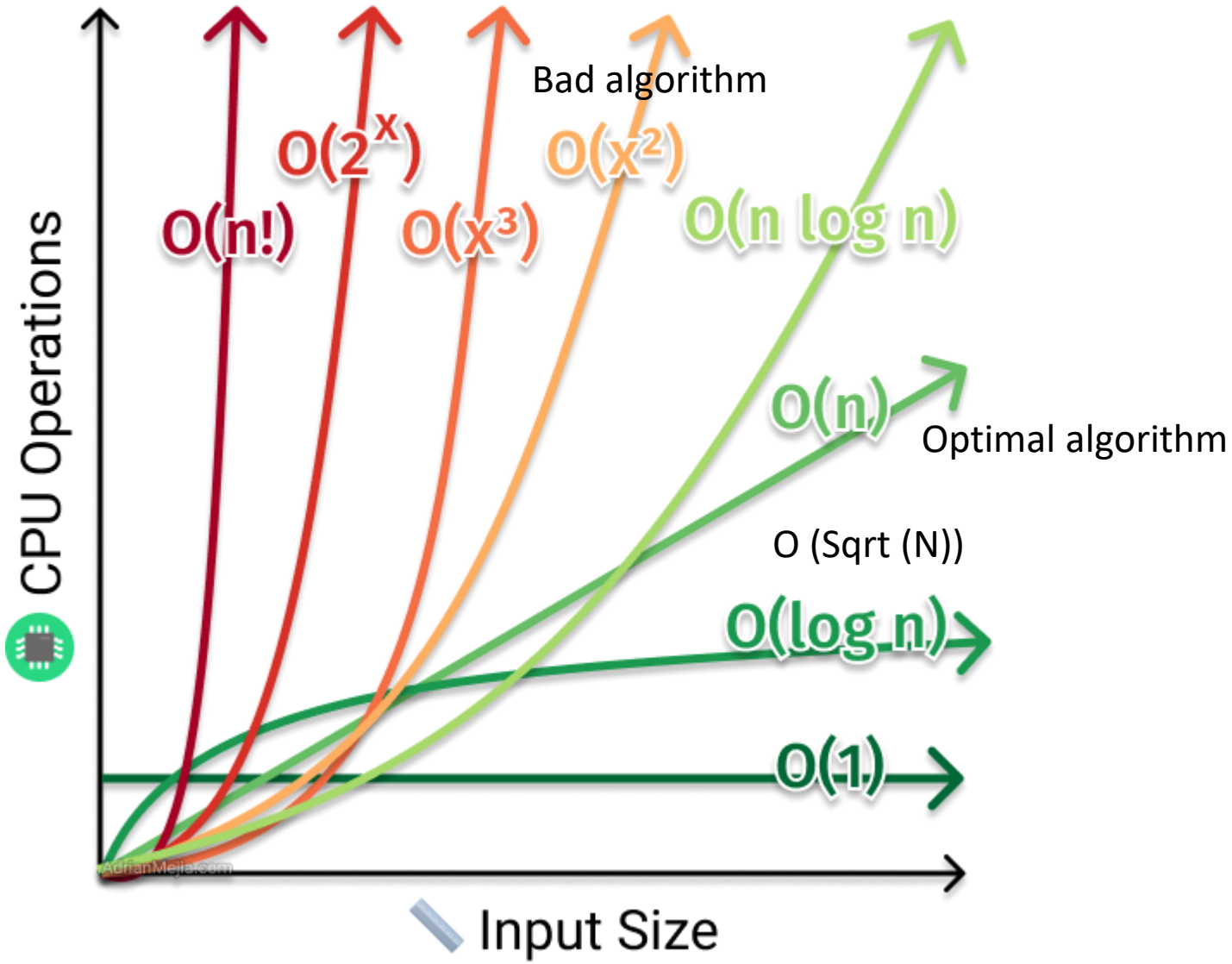
Space Complexity: Amount of memory needed relative to input size.

Focus on how complexity changes with growing input.

Types of Asymptotic Notations

Big O (O) → Worst case (Upper bound). (When the answer is found **at the end**.)---- We will use this to find T&S Complexity
Omega (Ω) → Best case (Lower bound). (When the answer is found **immediately**.)
Theta (Θ) → Average case (Tight bound).

⏱️ Time Complexity



Time Complexity	Notation	Example Algorithm	Description
Constant	O(1)	Accessing an array element	Same time, no matter the input size
Logarithmic	O(log n)	Binary Search	Problem size is halved each step
Linear	O(n)	Linear Search	Goes through each element once
Linearithmic	O(n log n)	Merge Sort, Quick Sort (avg)	Divide and conquer with combining
Quadratic	$O(n^2)$	Bubble Sort, Selection Sort	Nested loops over input
Cubic	$O(n^3)$	Matrix multiplication (naive)	Triple nested loops
Exponential	$O(2^n)$	Recursive Fibonacci	Grows very fast, only for small inputs
Factorial	O(n!)	Travelling Salesman (Brute)	Extremely slow for large n

$O(1)$ constant: the operation doesn't depend on the size of its input, e.g. adding a node to the tail of a linked list where we always maintain a pointer to the tail node.

$O(n)$ linear: the run time complexity is proportionate to the size of n.

$O(log\ n)$ logarithmic: normally associated with algorithms that break the problem into smaller chunks per each invocation, e.g. searching a binary search tree.

$O(n\ log\ n)$ just $n\ log\ n$: usually associated with an algorithm that breaks the problem into smaller chunks per each invocation, and then takes the results of these smaller chunks and stitches them back together, e.g. quick sort.

$O(n^2)$ quadratic: e.g. bubble sort.

$O(n^3)$ cubic: very rare.

$O(2^n)$ exponential: incredibly rare.

In algorithm design, there's often a trade-off between optimizing for time complexity (execution speed) and space complexity (memory usage).

1)a statement involving basic operations

Here are some examples of basic operations.

- -one arithmetic operation (eg., +, *)
- -one assignment
- -one test (eg., x==0)
- -one read(accessing an element from an array)

2) Sequence of statements involving basic operations.

```
statement 1;
statement 2;
.....
statement k;
```

1. Traversing an array.
2. Sequential/Linear search in an array.
3. Best case time complexity of Bubble sort (i.e when the elements of array are in sorted order).

Basic strucure is :

```
for (i = 0; i < N; i++) {
    sequence of statements of O(1)
}
```

1) Worst case time complexity of Bubble, Selection and Insertion sort.

Nested loops:

1.

```
for (i = 0; i < N; i++) {
    for (j = 0; j < M; j++) {
        sequence of statements of O(1)
    }
}
```

```
for (i = 0; i < N; i++) {
    for (j = 0; j < N; j++) {
        sequence of statements of O(1)
    }
}
```

```
for (i = 0; i < N; i++) {
    for (j = i+1; j < N; j++) {
        sequence of statements of O(1)
    }
}
```

Let us see how many iterations the inner loop has:

Value of i	Number of iterations of inner loop
0	N-1
1	N-2
.....	.....
N-3	2
N-2	1
N-1	0

So the total number of times the "sequence of statements" within the two loops executes is :
(N-1)+(N-2)+.....2+1+0 which is N*(N-1)/2 or (1/2)*(N^2) - (1/2)* N
and we can say that it is O(N^2) (we can ignore multiplicative constant and for large problem size the dominant term determines the time complexity)

Let us see how many iterations the inner loop has:

Value of i	Number of iterations of inner loop
0	N-1
1	N-2
.....	.....
N-3	2
N-2	1
N-1	0

So the total number of times the "sequence of statements" within the two loops executes is :
(N-1)+(N-2)+.....2+1+0 which is N*(N-1)/2 or (1/2)*(N^2) - (1/2)* N
and we can say that it is O(N^2) (we can ignore multiplicative constant and for large problem size the dominant term determines the time complexity)

•Use platforms like LeetCode, HackerRank, Codeforces.

BCSE202L DSA Fall25_26

WhatsApp group

Analysis of Recursive algorithms

Recursive Algorithm Analysis

- A recursive function is a function that calls itself within its own definition. It's a way to solve problems by breaking them down into smaller, self-similar subproblems until a base case is reached, which stops the recursion.
- Analyzing them usually requires solving a **recurrence relation**.

```c
int binarySearch(int arr[], int low, int high, int key) {
    if (low > high) return -1;
    int mid = (low + high)/2;
    if (arr[mid] == key) return mid;
    else if (arr[mid] > key)
        return binarySearch(arr, low, mid - 1, key);
    else
        return binarySearch(arr, mid + 1, high, key);
}
```

C

```c
#include <stdio.h>

// Recursive function to calculate factorial
long long factorial(int n) {
    // Base Case: Factorial of 0 is 1
    if (n == 0) {
        return 1; // Constant time operation
    }
    // Recursive Step
    else {
        return n * factorial(n - 1); // Multiplication + recursive call
    }
}

int main() {
    int num = 5;
    printf("Factorial of %d is %lld\n", num, factorial(num));
    return 0;
}
```

Analysis of Non-Recursive algorithms

Non-Recursive Algorithm Analysis
Non-recursive algorithms do **not call themselves**.
It analyze them by **counting loops, operations, or steps**

```c
int findMax(int arr[], int n) {
    int max = arr[0];
    for(int i = 1; i < n; i++) {
        if(arr[i] > max)
            max = arr[i];
    }
    return max;
}
```

int arr[] = {3, 7, 2, 9, 1, 6};
int n = 6;

Step	i	arr[i]	max before	Comparison	max after
Init	—	—	—	—	max = 3
1	1	7	3	7 > 3 → true	max = 7
2	2	2	7	2 > 7 → false	max = 7
3	3	9	7	9 > 7 → true	max = 9
4	4	1	9	1 > 9 → false	max = 9
5	5	6	9	6 > 9 → false	max = 9

- The loop runs from i = 1 to i = n-1 → Total comparisons: **n-1**
- So time complexity: **O(n)**

Asymptotic analysis aims to find the "Big-O" complexity of this relation, how the runtime grows as the input size n becomes very large.

It don’t measure the actual running time

But How the time (or space) taken by an algorithm increases with the input size is also calculated.

Here are the four methods to solve these relations:

1.Iteration Method

2.Recursive Tree Method

3.Substitution Method

4.Master Method

Guidelines to Asymptotic Analysis

There are some general rules to help us determine the running time of an algorithm.

1) **Loops**: The running time of a loop is, at most, the running time of the statements inside the loop (including tests) multiplied by the number of iterations.

```
// executes n times
for (i=1; i<=n; i++)
    m = m + 2; // constant time, c
```

Total time = a constant c × n = c n = O(n).

2) **Nested loops**: Analyze from the inside out. Total running time is the product of the sizes of all the loops.

```
//outer loop executed n times
for (i=1; i<=n; i++) {
    // inner loop executes n times
    for (j=1; j<=n; j++)
        k = k+1; //constant time
}
```

Total time = c × n × n = cn^2 = $O(n^2)$.

3) **Consecutive statements**: Add the time complexities of each statement.

```
#include <stdio.h>
int main() {
    int x = 0, m = 0, k = 0;
    int n = 10; // You can change n for testing

    // Constant time
    x = x + 1;

    // First loop: executes n times → O(n)
    for (int i = 1; i <= n; i++) {
        m = m + 2;  // constant time inside loop
    }

    // Nested loop: O(n^2)
    for (int i = 1; i <= n; i++) {
        for (int j = 1; j <= n; j++) {
            k = k + 1;  // constant time inside inner loop
        }
    }
    printf("Final values: x = %d, m = %d, k = %d\n", x, m, k);
    return 0;
}
```

Total time = $c_0 + c_1 n + c_2 n^2 = O(n^2)$.

4) **If-then-else statements**: Worst-case running time: the test, plus either the then part or the else part (whichever is the larger).

```
#include <stdio.h>
#include <stdbool.h> // for using true/false

// Function to compare two integer arrays
bool compareLists(int list[], int otherList[], int length) {
    // test: constant time
    if (length == 0) {
        return false; // then-part: constant
    } else {
        // else-part: O(n) loop
        for (int i = 0; i < length; i++) {
            // constant-time comparison
            if (list[i] != otherList[i]) {
                return false;
            }
        }
    }
    return true; // If all elements matched
}
```

```
int main() {
    int list[] = {1, 2, 3, 4, 5};
    int otherList[] = {1, 2, 3, 4, 5};
    int length = sizeof(list) / sizeof(list[0]);

    if (compareLists(list, otherList, length)) {
        printf("Lists are equal.\n");
    } else {
        printf("Lists are not equal.\n");
    }

    return 0;
}
```

c_0 = test time, c_1 = then-part, c_2, c_3 = per-iteration operations in else-part

$$\text{Total time} = c_0 + c_1 + (c_2 + c_3) * n = O(n).$$

5. Logarithmic complexity: An algorithm is O(logn) if it takes a constant time to cut the problem size by a fraction (usually by ½). As an example let us consider the following program:

```c
#include <stdio.h>

int main() {
    int n = 100;
    int i;

    for (i = 1; i <= n; i = i * 2) {
        printf("%d ", i);
    }

    return 0;
}
```

```c
#include <stdio.h>

int main() {
    int n = 100;
    int count = 0;

    while (n > 0) {
        n = n / 2;
        count++;
    }

    printf("Number of bits required: %d\n", count);
    return 0;
}
```

$$log(2^k) = logn$$
$$klog2 = logn$$
$$k = logn$$

Total time = O(*logn*).

Master theorem

$$T(n) = aT(n/b) + \theta(n^k \log^p n)$$

where n = size of the problem
a = number of subproblems in the recursion and a >= 1
n/b = size of each subproblem
b > 1, k >= 0 and p is a real number.

Then,

1.if $a > b^k$, then $T(n) = \theta(n^{\log_b a})$

2.if $a = b^k$, then
(a) if $p > -1$, then $T(n) = \theta(n^{\log_b a} \log^{p+1} n)$
(b) if $p = -1$, then $T(n) = \theta(n^{\log_b a} \log.\log n)$
(c) if $p < -1$, then $T(n) = \theta(n^{\log_b a})$

3.if $a < b^k$, then
(a) if $p >= 0$, then $T(n) = \theta(n^k \log^p n)$
(b) if $p < 0$, then $T(n) = \theta(n^k)$

Master theorem

1. $T(n) = 3T(n/2) + n^2$
2. $T(n) = 4T(n/2) + n^2$
3. $T(n) = T(n/2) + n^2$
4. $T(n) = 2^nT(n/2) + n^2$
5. $T(n) = 16T(n/4) + n^{0.85}$
6. $T(n) = 3T(n/2) + \log^2 n$
7. $T(n) = 2T(n/2) + n\log^2 n$

Master theorem

Problem-1 $T(n) = 3T(n/2) + n^2$

Solution: $T(n) = 3T(n/2) + n^2 => T(n) = \Theta(n^2)$ (Master Theorem Case 3.a)

Problem-2 $T(n) = 4T(n/2) + n^2$

Solution: $T(n) = 4T(n/2) + n^2 => T(n) = \Theta(n^2 logn)$ (Master Theorem Case 2.a)

Problem-3 $T(n) = T(n/2) + n^2$

Solution: $T(n) = T(n/2) + n^2 => \Theta(n^2)$ (Master Theorem Case 3.a)

Problem-4 $T(n) = 2^n T(n/2) + n^n$

Solution: $T(n) = 2^n T(n/2) + n^n =>$ Does not apply (a is not constant)

Problem-5 $T(n) = 16T(n/4) + n$

Solution: $T(n) = 16T(n/4) + n => T(n) = \Theta(n^2)$ (Master Theorem Case 1)

Master theorem

Problem-7 $T(n) = 2T(n/2) + n/logn$

Solution: $T(n) = 2T(n/2) + n/logn => T(n) = \Theta(nloglogn)$ (Master Theorem Case 2. b)

Problem-8 $T(n) = 2T(n/4) + n^{051}$

Solution: $T(n) = 2T(n/4) + n^{051} => T(n) = \Theta(n^{0.51})$ (Master Theorem Case 3.b)

Problem-9 $T(n) = 0.5T(n/2) + 1/n$

Solution: $T(n) = 0.5T(n/2) + 1/n$ => Does not apply ($a < 1$)

Problem-10 $T(n) = 6T(n/3) + n^2 logn$

Solution: $T(n) = 6T(n/3) + n^2logn => T(n) = \Theta(n^2logn)$ (Master Theorem Case 3.a)

Problem-11 $T(n) = 64T(n/8) - n^2logn$

Solution: $T(n) = 64T(n/8) - n^2logn$ => Does not apply (function is not positive)

Problem-12 $T(n) = 7T(n/3) + n^2$

Solution: $T(n) = 7T(n/3) + n^2 => T(n) = \Theta(n^2)$ (Master Theorem Case 3.as)

Commonly used Logarithms and Summations

Logarithms

$$\log x^y = y \log x \qquad \log n = \log_{10}^{n}$$

$$\log xy = \log x + \log y \qquad \log^k n = (\log n)^k$$

$$\log \log n = \log(\log n) \qquad \log \frac{x}{y} = \log x - \log y$$

$$a^{\log_b^x} = x^{\log_b^a} \qquad \log_b^x = \frac{\log_a^x}{\log_a^b}$$

Arithmetic series

$$\sum_{K=1}^{n} k = 1 + 2 + \cdots + n = \frac{n(n+1)}{2}$$

Geometric series

$$\sum_{k=0}^{n} x^k = 1 + x + x^2 \ldots + x^n = \frac{x^{n+1} - 1}{x - 1} (x \neq 1)$$

Harmonic series

$$\sum_{k=1}^{n} \frac{1}{k} = 1 + \frac{1}{2} + \ldots + \frac{1}{n} \approx \log n$$

Other important formulae

$$\sum_{k=1}^{n} \log k \approx n \log n$$

$$\sum_{k=1}^{n} k^p = 1^p + 2^p + \cdots + n^p \approx \frac{1}{p+1} n^{p+1}$$